

Họ tên: Nguyễn Anh Quân

Ngày 04/08/2026

Mục Lục

1. Git flow là gì ?	1
2. Git Convention (quy ước đặt tên & commit)	3
2.1. Quy ước đặt tên nhánh (Branch Naming convention)	3
2.2. Quy ước commit message (Conventional Commits)	4
2.3. Lợi ích khi áp dụng	6
3. Các quy trình làm việc với Git	6
3.1. Mô hình GitHub Flow	6
3.2. Mô hình Trunk-based Development	7
3.3. Mô hình Forking Workflow	9
3.4. Bảng so sánh nhanh	10
4. Các khái niệm quan trọng khác	10
4.1. Pull Request / Merge Request (PR/MR)	10
4.2. Rebase vs Merge	11
4.3. File .gitignore	13
4.4. Semantic Versioning (SemVer)	14
4.5. Conflict Resolution (giải quyết xung đột khi merge)	14

TÌM HIỂU VỀ GIT FLOW, GIT CONVENTION VÀ CÁC QUY TRÌNH LÀM VIỆC VỚI GIT

1. Git flow là gì ?

Git flow là một mô hình quản lý branch có quy định rõ ràng vai trò của từng nhánh trong vòng đời phát triển của sản phẩm:

- **Nhánh main/master:** Đây là nhánh mà code đã được release, luôn ở trạng thái ổn định, production-ready (luôn sẵn sàng cho người dùng).
- **Nhánh develop:** Nhánh tích hợp, chứa code mới nhất đã merge, chuẩn bị cho release tiếp theo.
- **Feature/xxx:** Nhánh được tạo từ nhánh **develop**, dùng để phát triển 1 tính năng cụ thể. Sau khi xong thì merge ngược lại vào **develop**.
- **Release/xxx:** Cũng là nhánh được tạo từ nhánh **develop**, nhưng nó được dùng để test, sửa bug nhỏ phát sinh, khác với **Feature** trên là hoàn toàn không thêm tính năng mới. Và sau khi fix xong thì merge vào **develop** và **main**.
- **Hotfix/xxx:** Được tạo từ nhánh **main** khi cần fix gấp các bug ở phía người dùng (sản phẩm). Xong thì merge vào cả **develop** và **main**.

Ý tưởng cốt lõi: Tách biệt code đang phát triển (nhánh **develop**) với code ổn định có thể đưa ra cho người dùng (nhánh **main**). Và mỗi khi cần thêm tính năng khi thêm mới (feature), hay giải quyết các bug cần fix (release, hotfix) đều sẽ có 1 branch riêng để không bị ảnh hưởng tới nhau.

Giải thích thực tế:

Develop: là cái “bàn làm việc chung”. Ai làm xong các tính năng gì cũng phải đem đề lên đây trước.

Feature/xxx: khi bạn được giao 1 task (ví dụ: làm chức năng đăng nhập) thì tôi sẽ không code trực tiếp lên nhánh **develop**. Mà tôi sẽ tách ra một nhánh riêng (**feature/login**), code và test commit tại nhánh đó thoải mái bao nhiêu tùy thích mà

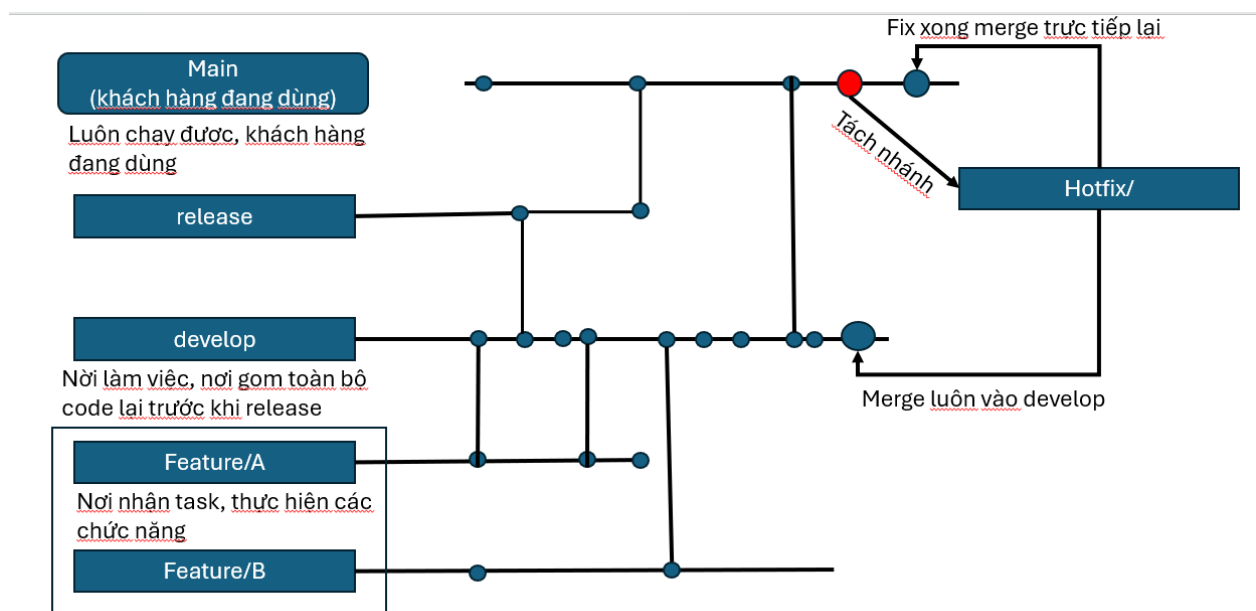
không sợ làm hỏng đến code của người khác trong suốt quá trình làm. Xong xuôi hết tất cả ỏn bao gồm cả test, thì tôi sẽ tạo PR và Merge PR (nếu không còn conflict).

Release/xxx: Khi nhánh **develop** đã đi các tính năng để chuẩn bị ra bản cập nhật mới, team sẽ tách nhánh **release/xxx** (ví dụ **release/1.1.0**) ra để rà soát lại lần cuối: test kĩ, sửa bug nhỏ hoặc phát sinh (tuyệt đối không thêm tính năng mới vì dễ sinh thêm bug mới). Sau khi xong thì merge nhánh vào cả **main** (chính thức) và **develop**.

Main: Đây là bản mà người dùng đang sử dụng và đang chạy. Và cũng chính là nhánh chỉ nhận code đã được kiểm chứng kĩ từ release (hoặc hotfix), không bao giờ code thử nghiệm (**develop**) được đẩy thẳng vào đây (**main**).

Hotfix/xxx: Ví dụ app đang chạy cho người dùng sử dụng thực tế mà phát hiện bug nghiêm trọng (ví dụ như: bảo mật, thông báo, các tính năng nghiêm trọng khác), thì không thể đợi đến đợt release tiếp theo được. Thì lúc này phải tách nhánh **hotfix/xxx** (**security, bug,...**) từ main (không phải từ dev, vì cần fix ngay trên bản đang chạy), sửa xong merge ngược vào **main** (để deploy fix ngay lập tức lỗi) và **develop** (để lần sau release không bị lỗi tiếp bug cũ).

Sơ đồ hình dung:



2. Git Convention (quy ước đặt tên & commit)

2.1. Quy ước đặt tên nhánh (Branch Naming convention)

Cấu trúc chung <Loại>/<Mô tả ngắn gọn>:

- Chữ thường toàn bộ, các từ cách nhau bằng dấu gạch ngang - (kebab – case), không dùng khoảng trắng, không dùng chữ hoa.
- Mô tả ngắn gọn nhưng đủ hiểu nội dung, tránh đặt tên chung chung như feature/fix hay feature/update.

Các loại nhánh phổ biến:

Prefix	Khi nào dùng	Ví dụ
feature/	Phát triển tính năng mới	feature/login-page feature/payment-integration
bugfix/	Sửa lỗi phát hiện trong quá trình dev/test (chưa lên production)	bugfix/wrong-total-price
hotfix/	Sửa lỗi khẩn cấp trên production	hotfix/fix-login-crash
release/	Chuẩn bị phát hành 1 phiên bản	release/1.1.0, release/2024-q2
chore/	Việc không liên quan trực tiếp tính năng (update dependency, config CI/CD,...)	chore/update-eslint-config
docs/	Cập nhật tài liệu	docs/update-readme

Một số team còn gắn thêm mã task/ticket (nếu dùng Jira, Trello...) để dễ trace:

- feature/PROJ-123-login-page
- bugfix/PROJ-456-total-price

Lợi ích: Nhìn nên tên nhánh biết ngay đang làm task nào, để liên kết ngược lại hệ thống quản lý công việc.

Vài quy tắc thêm:

- Không đặt tên nhánh trùng với tên đã bị xoá trước đó (để gây nhầm lẫn khi tra lịch sử).
- Nhánh **feature**/, **bugfix**/ nên xoá sau khi đã merge xong để tạo repo không bị rác.
- Không commit trực tiếp lên **main/develop** – luôn làm qua nhánh riêng rồi tạo Pull Request/Merge Request.

2.2. Quy ước commit message (Conventional Commits)

Cấu trúc đầy đủ:

- **Tiêu đề:** Mô tả ngắn gọn
- **Mô tả chi tiết:** Tùy chọn có thể viết cụ thể ra
- **Footer:** tùy chọn (breaking change, liên kết,...)

Ví dụ:

- Feature(auth): Add JWT login validation
- Add middleware to verify JWT token before accessing protected routes, at <where/link/instruction>. Token expiry set to 24h.
- Closes #123

Bảng type đầy đủ và dùng

Type	Ý nghĩa	Ví dụ
feat	Thêm tính năng mới	feat(cart): add remove item button
fix	Sửa lỗi	fix(auth): fix token not refreshed after expiry
docs	Chỉ thay đổi tài liệu (README, comment,...)	docs(api): update endpoint documentation
style	Format code (thụt lề, dấu chấm phẩy,...), không đổi logic	style: format code with prettier

refactor	Tái cấu trúc code, không thêm tính năng, không sửa lỗi chức năng	refactor(user): simplify validation logic
perf	Cải thiện hiệu năng	perf(query): optimize database indexing
test	Thêm/ sửa test	test(auth): add unit test for login API
chore	Việc phụ trợ (update dependency, config build,...)	chore: update axios to v1.6.0
build	Thay đổi liên quan hệ thống build, CI/CD	build: update Dockerfile
revert	Revert lại 1 commit trước đó	revert: revert "feat(auth): add JWT login"

Phân <phạm vi> (scope) – tùy chọn (có thể có hoặc không, nhưng khuyến khích có)

- Cho biết commit ảnh hưởng đến các module/khu vực nào của code, ví dụ như:auth, cart, api, ui, db,...

fix(cart): Fix quantily not updating when click + feat (profile): add avt upload

- Quy tắc viết mô tả ngắn
 - Dùng động từ nguyên mẫu, thì hiện tại (giống ra lệnh): “add”, “fix”, “update” – không dùng “added”, “fixed”,...
 - Không viết hoa chữ đầu, không chấm câu cuối dòng.
 - Giới hạn khoảng 50-70 kí tự cho dòng đầu tiên.
- Đúng: fix(login): fix crash when password field empty
- Sai: Fixed bug. (Không rõ sửa gì, viết hoa, dùng tense để viết commit).
- Breaking Change

Nếu commit làm thay đổi phá vỡ tương thích ngược, thêm “!” sau type/scope hoặc ghi rõ trong footer:

- feat(api)!: change response format of /users endpoint

- Push lên, tạo Pull Request (PR) vào main.
- Team review code, chạy CI (test tự động lint,..) ngay trên PR.
- PR được duyệt merge vào main, nếu có conflict thì resolve
- Main merge xong là deploy ngay lập tức (vì main luôn ở trong trạng thái sẵn sàng release).

Đặc điểm:

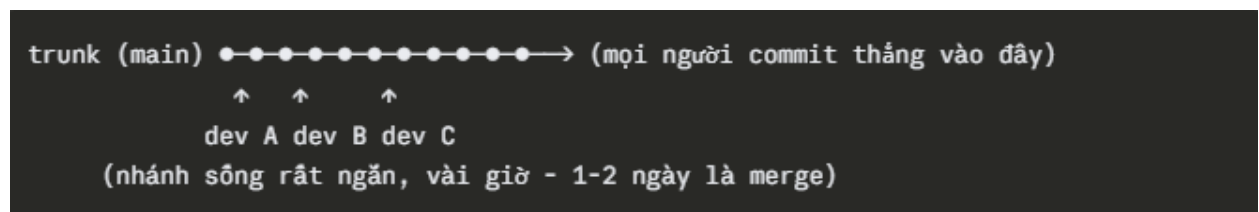
- **Đơn giản hơn Git Flow rất nhiều** - không cần nhớ 5 loại nhánh, không cần các bước merge qua lại giữa release/develop/main.
- **Chu kỳ release cực nhanh:** có thể deploy nhiều lần 1 ngày.
- Phù hợp với các app web/service dùng **CI/CD**, nơi mỗi lần merge PR là tự động build & deploy (vd: SaaS, web app cập nhật liên tục).
- **Yêu cầu bắt buộc:** phải có bộ test tự động tốt, vì không có bước "release" riêng để rà soát kỹ như Git Flow - nếu code lỗi merge vào main là ảnh hưởng production ngay.

Nên dùng khi:

- Team nhỏ, sản phẩm là web app/service deploy liên tục
- Không cần quản lý nhiều phiên bản cùng lúc (không giống app mobile phải support version cũ).

3.2. Mô hình Trunk-based Development

Mô hình:



Đặc điểm:

- Tất cả dev đều làm việc trên (gần như) cùng 1 nhánh chính gọi là trunk (thường chính là main)
- Nếu có tách nhánh riêng thì nhánh đó sống rất ngắn - chỉ vài giờ đến 1-2 ngày, code xong là merge liền, không để nhánh tồn tại lâu như feature/ trong Git Flow.
- Vì code được đẩy vào trunk rất thường xuyên, kể cả khi tính năng chưa hoàn thiện, nên bắt buộc dùng Feature Flag (cờ bật/tắt tính năng) để ẩn tính năng chưa xong khỏi người dùng thật, mà code vẫn nằm trên trunk.

Ví dụ feature flag:

```
if (featureFlags.newCheckoutFlow) {  
  
    showNewCheckout();  
  
} else { showOldCheckout();  
  
}
```

Code đã merge vào trunk nhưng tính năng newCheckoutFlow đang tắt, khi nào sẵn sàng chỉ cần bật flag lên (không cần merge code mới).

Lợi ích:

- Tránh "merge hell": nhánh sống càng lâu, càng lệch xa main, càng dễ conflict khi merge. Trunk-based giảm thiểu việc này tối đa vì merge liên tục.
- Tích hợp code liên tục -> phát hiện lỗi/conflict sớm.
- Rất hợp với các team làm CI/CD ở mức độ cao (deploy hàng chục lần/ngày), các công ty lớn như Google, Facebook dùng mô hình này.

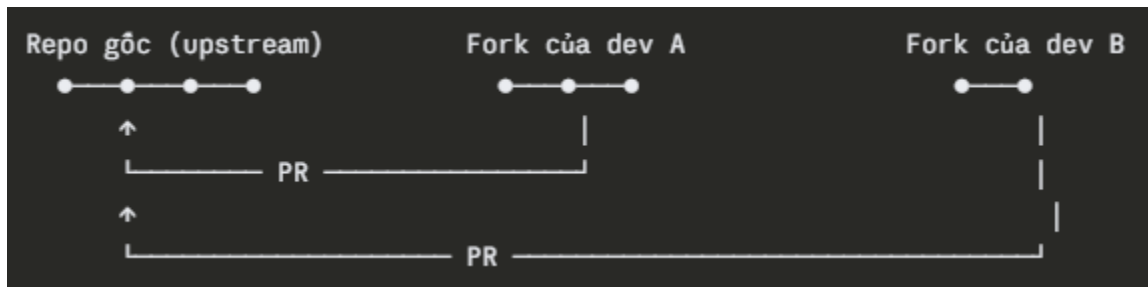
Nhược điểm – thách thức:

- Đòi hỏi kỷ luật cao: code chưa xong vẫn phải đảm bảo không làm sập trunk.
- Cần hạ tầng feature flag, test tự động rất mạnh

- Không phù hợp team ít kinh nghiệm hoặc chưa có CI/CD ổn định.

3.3. Mô hình Forking Workflow

Mô hình:



Quy trình:

- Dev fork (tạo bản sao riêng, thuộc tài khoản cá nhân) repo gốc về tài khoản của mình.
- Clone bản fork đó về máy, code, commit thoải mái trên fork của mình - không đụng gì đến repo gốc.
- Push code lên fork của mình.
- Tạo Pull Request từ fork cá nhân -> repo gốc.
- Chủ repo gốc (maintainer) review, nếu ok thì merge vào repo gốc.

Đặc điểm:

- Người đóng góp không cần được cấp quyền write vào repo gốc - đây là điểm khác biệt lớn nhất so với Git Flow/GitHub Flow (ở 2 mô hình kia, dev cần quyền tạo nhánh trực tiếp trên repo chung).
- Repo gốc hoàn toàn kiểm soát code nào được merge vào, an toàn hơn khi có hàng trăm/ngày nghìn người lạ muốn đóng góp.

Nên dùng khi:

- Dự án mã nguồn mở (open-source): ai cũng có thể đóng góp mà không cần xin quyền trước, ví dụ các repo lớn trên GitHub (React, VS Code...).
- Cũng có thể dùng trong công ty khi muốn tách biệt rõ ràng giữa "code thử nghiệm cá nhân" và "code chính thức của team", dù ít phổ biến hơn trong nội bộ doanh nghiệp.

3.4. Bảng so sánh nhanh

Tiêu chí	Git Flow	GitHub Flow	Trunk-based	Forking
Độ phức tạp	Cao	Thấp	Trung bình (cần kỹ luật)	Trung bình
Số nhánh chính	2 (main, develop)	1 (main)	1 (trunk)	Không giới hạn(mỗi người 1 fork)
Nhánh phụ	feature/release/hotfix	feature	Rất ngắn hạn	toàn bộ code nằm trên fork
Tốc độ release	Chậm hơn, theo chu kỳ	Rất nhanh, liên tục	Rất nhanh, liên tục	Tuỳ maintainer
Phù hợp	App có versioning rõ ràng (mobile app...)	Web/SaaS deploy liên tục	Team lớn, CI/CD mạnh	Open-source
Yêu cầu hạ tầng	Trung bình	Cần CI/CD tốt	Cần CI/CD + feature flag mạnh	Không đặc biệt

4. Các khái niệm quan trọng khác

4.1. Pull Request / Merge Request (PR/MR)

Là một yêu cầu “xin merge” code từ nhánh của bạn vào nhánh đích (thường là **develop** hoặc **main**), kèm theo cơ chế review bắt buộc trước khi merge. **GitHub** gọi là **Pull Request**, **GitLab** gọi là **Merge Request** – bản chất giống nhau.

Quy trình thường gặp:

- Push nhánh feature/xxx lên remote.
- Tạo PR, ghi rõ: tiêu đề, mô tả thay đổi, liên kết tới ticket/issue (nếu có), ảnh chụp màn hình nếu có UI thay đổi
- Hệ thống CI tự động chạy: build, lint, unit test... trên PR đó.
- Reviewer (1 hoặc nhiều người) đọc code, để lại comment, yêu cầu sửa (nếu có), approve khi ok.
- Sau khi đủ điều kiện (CI pass + đủ approve), mới được merge.

Vì sao quan trọng:

- Là "cửa kiểm soát chất lượng" bắt buộc - không ai được tự ý đẩy code lỗi vào nhánh chung.
- Tạo cơ hội học hỏi qua review: người khác đọc code sẽ phát hiện lỗi logic, gợi ý cách viết tốt hơn.
- Lưu lại lịch sử thảo luận (comment trên PR) - sau này ai cần hiểu vì sao code viết vậy có thể đọc lại.

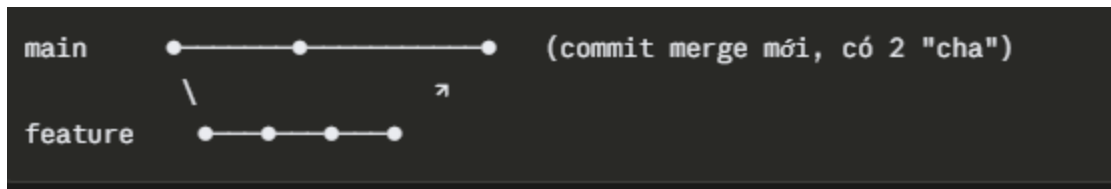
Một vài quy tắc PR tốt nên biết:

- PR nên nhỏ, tập trung 1 việc - PR quá lớn (sửa cả nghìn dòng) rất khó review kỹ.
- Tiêu đề PR nên theo convention giống commit (feat: add login page).
- Không tự merge PR của chính mình nếu team có quy định cần người khác approve.

4.2. Rebase vs Merge

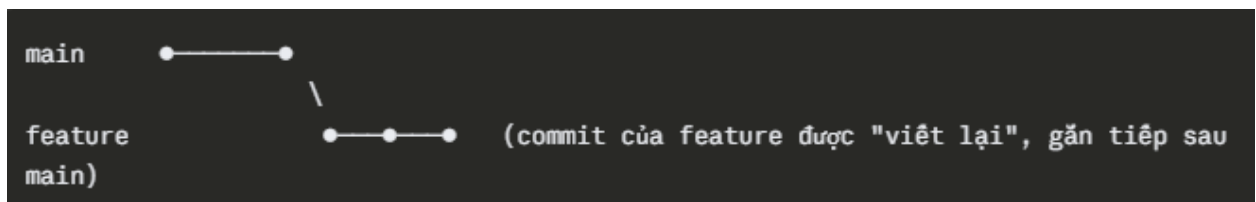
Cả hai đều dùng để đưa code từ nhánh này vào nhánh khác, nhưng cách hoạt động và kết quả lịch sử khác nhau hoàn toàn.

Merge:



- Giữ nguyên toàn bộ lịch sử commit của cả 2 nhánh.
- Tạo ra 1 merge commit mới (có 2 commit cha) để đánh dấu điểm gộp.
- Lệnh: `git merge feature/xxx`
- Ưu điểm: an toàn, không đổi lịch sử cũ, dễ trace lại được commit nào thuộc nhánh nào.
- Nhược điểm: lịch sử (`git log`) trông rối, có nhiều merge commit chằng chịt nếu nhiều người merge liên tục.

Rebase:



- "Xếp lại" các commit của nhánh feature lên trên đỉnh mới nhất của main - như thể bạn tạo nhánh feature từ bây giờ chứ không phải từ trước.
- Lệnh: `git rebase main` (đứng ở nhánh feature)
- Ưu điểm: lịch sử commit thẳng hàng, sạch, dễ đọc như 1 đường thẳng, không có merge commit rác.
- Nhược điểm quan trọng: rebase viết lại commit hash (tạo commit mới thay cho commit cũ) -> nếu nhánh đó đã push lên remote và người khác đang pull về dùng chung, rebase sẽ làm lịch sử của họ và bạn lệch nhau, dễ gây lỗi nghiêm trọng khi push lại (force push).

Quy tắc an toàn kinh điển:

Không rebase nhánh đã chia sẻ (shared) với người khác. Chỉ rebase nhánh cá nhân, chưa push, hoặc chỉ mình bạn đang làm việc trên đó.

Khi nào dùng cái nào:

- **Rebase:** khi làm việc cá nhân trên feature/ của riêng bạn, muốn đồng bộ code mới nhất từ develop về trước khi tạo PR, để lịch sử sạch.
- **Merge:** khi merge PR vào nhánh chung (develop, main) — hầu hết công cụ Git server (GitHub, GitLab) mặc định dùng merge (hoặc "squash merge" - gộp hết commit của PR thành 1 commit duy nhất) để giữ nhánh chính an toàn, dễ trace.

4.3. File .gitignore

Là file cấu hình liệt kê các file/thư mục mà Git sẽ bỏ qua, không track, không track, không cho phép commit (trừ khi ép buộc).

Mục đích: tránh commit nhầm các file:

- File build/output tự sinh ra: node_modules/, dist/, build/, bin/, obj/
- File cấu hình cá nhân, chứa các thông tin nhạy cảm: .env, appsettings.Development.json.
- File của IDE/OS: .vscode/, .idea/, .DS_Store

Ví dụ như file: .gitignore:

node_modules/

.env

*.log

build/

.vscode/

Vì sao quan trọng:

- Giữ repo gọn nhẹ, không commit hàng nghìn file thư viện không cần thiết.
- Bảo mật: tránh push nhằm mật khẩu, API key, connection string lên Git (đặc biệt nguy hiểm nếu repo public).
- Lưu ý: nếu file đã lỡ được commit trước đó, thêm vào .gitignore sau này không tự động xoá khỏi lịch sử — phải dùng lệnh riêng (git rm --cached) để gỡ.

4.4. Semantic Versioning (SemVer)

Cấu trúc MAJOR.MINOR.PATCH

Thành phần	Tăng khi	Ví dụ
MAJOR	Có thay đổi phá vỡ tương thích ngược (breaking change) – code cũ dùng bản mới có thể lỗi	1.x.x -> 2.0.0
MINOR	Thêm tính năng mới nhưng vẫn tương thích ngược	1.2.x -> 1.3.0
PATCH	Sửa lỗi nhỏ, không thêm tính năng, không phá vỡ gì	1.2.3 -> 1.2.4

Liên hệ với commit convention:

- **Commit fix:** -> tăng PATCH.
- **Commit feat:** -> tăng MINOR.
- **Commit có BREAKING CHANGE** -> Tăng MAJOR.

Đây là lý do các tool tự động (semantic-release) có thể tự tính version tiếp theo dựa vào commit message, không cần con người tự quyết định thủ công.

4.5. Conflict Resolution (giải quyết xung đột khi merge)

Xảy ra khi 2 nhánh cùng sửa cùng **1 dòng/cùng 1 vùng code khác nhau**, Git không tự biết nên giữ bản nào.

Git sẽ đánh dấu trong file bị conflict như sau:

```
<<<<<<< HEAD (nhánh hiện tại - vd develop)
const price = product.price * 1.1;
=====
const price = product.price * quantity;
>>>>>> feature/discount (nhánh đang merge vào)
```

Cách xử lí:

- Mở file bị conflict, đọc kỹ cả 2 phần để hiểu ý đồ của từng bên.
- Chỉnh sửa lại thành 1 đoạn code đúng (có thể giữ 1 trong 2, hoặc gộp cả 2, hoặc viết lại hoàn toàn).
- Xoá các dấu <<<<<<<, =====, >>>>>>>.
- git add file đã sửa, rồi git commit (nếu đang merge) hoặc git rebase --continue (nếu đang rebase).

Cách hạn chế conflict:

- Pull/merge code mới từ nhánh chính về nhánh của mình thường xuyên, đừng để nhánh sống quá lâu rồi mới đồng bộ (liên hệ lại lý do Trunk-based Development tránh được conflict lớn).
- Chia task nhỏ, tránh nhiều người cùng sửa 1 file trong cùng thời điểm.
- Giao tiếp trong team khi biết sắp đụng vào cùng 1 khu vực code.